Vision Transformer (ViT) for Image Classification

Land-Use Classification of Satellite Imagery Using PyTorch, timm, ONNX and FastAPI

TABLE OF CONTENTS

- **1. INTRODUCTION**
  - 1.1 Overview
  - 1.2 Purpose
- **2. LITERATURE SURVEY**
- **3. THEORETICAL ANALYSIS**
  - 3.1 Block diagram
  - 3.2 Hardware / Software designing
- **4. EXPERIMENTAL INVESTIGATION**
  - 4.1 Background
  - 4.2 Theoretical Analysis
- **5. FLOWCHART**
  - 5.1 Use Case
  - 5.2 Collaboration Diagram
- **6. RESULT**
- **7. ADVANTAGES & DISADVANTAGES**
- **8. APPLICATIONS**
- **9. CONCLUSION**
- **10. FUTURE SCOPE**
- **11. BIBLIOGRAPHY**
- **APPENDIX**
  - A. Source code

LIST OF FIGURES

- Figure 3.1 Block diagram

- Figure 4.1.1 Vision Transformer architecture
- Figure 5.1 Use case diagram
- Figure 5.2 Collaboration diagram
- Figure 6.1 Training and validation curves
- Figure 6.2 Confusion matrix on the test set
- Figure 6.3 Grad-CAM visualisation

ABSTRACT

This project presents an end-to-end Vision Transformer (ViT) system for image classification, built with PyTorch and exposed as a REST API. Instead of relying on convolutions alone, a ViT splits every image into fixed-size patches, treats the patches as a sequence of tokens, and uses multi-head self-attention to model the relationship between every pair of patches, capturing both local detail and global context. The project implements a ViT from scratch and also fine-tunes a pretrained ViT-S/16 (through the timm library) on EuroSAT, a dataset of 27,000 Sentinel-2 satellite tiles spanning ten land-use classes. A ResNet-50 baseline is trained with the same pipeline so that the comparison is fair.

Modern training techniques are used throughout: RandAugment, RandomErasing, Mixup and CutMix, label smoothing, stochastic depth, the AdamW optimizer with warm-up and cosine learning-rate decay, gradient clipping and mixed-precision training. The models are evaluated with accuracy, macro F1-score, per-class reports and confusion matrices, interpreted with Grad-CAM heat-maps, exported to the ONNX format, and served through a FastAPI endpoint and a Gradio interface. The system can be applied in land-use monitoring, urban planning, agriculture, environmental protection and disaster response, and the same pipeline can be reused for any image-classification dataset.

Language used: Python

Project Domain: Artificial Intelligence (Computer Vision)

---

1. INTRODUCTION

1.1 Overview

The Vision Transformer project is a complete image-classification pipeline written in Python. It takes an input image, converts it into a sequence of patch embeddings, processes the sequence with a stack of Transformer encoder blocks, and predicts the class of the image from a special classification ([CLS]) token. The same code base supports three interchangeable models: a ViT written from scratch, a pretrained ViT-S/16 that is fine-tuned on the target data, and a ResNet-50 convolutional baseline.

Image classification is the foundation of many applications, from medical diagnosis to autonomous driving. For almost a decade Convolutional Neural Networks (CNNs) were the default architecture. The introduction of the Vision Transformer showed that an architecture designed for language can match or beat CNNs on vision tasks when it is trained at scale or initialised from pretrained weights. This project extends an earlier ViT implementation on CIFAR-10 (TensorFlow/Keras, 500-image subset) into a complete, modern workflow: a real-world dataset with a stratified train/validation/test split, a stronger training recipe, a CNN baseline, explainability, model export and deployment.

The data used is EuroSAT, a benchmark of 64 x 64 pixel RGB patches cut from Sentinel-2 satellite images of European cities. It has ten classes: AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River and SeaLake. Training is controlled by YAML configuration files, results are written to a run folder (model checkpoint, metrics, confusion matrix, logs), and the best model is exported to ONNX so that it can run on a CPU without PyTorch. A FastAPI service and a Gradio web interface make the trained model available to non-technical users.

1.2 Purpose

The purpose of the project is to build a reproducible, explainable and deployable ViT-based classifier, and to show how the original ViT idea can be combined with current engineering practice. Its key use cases include:

1. **Land-use and Land-cover Monitoring:** classifying satellite tiles to track how land is used and how it changes over time.
2. **Urban Planning:** identifying residential, industrial and highway areas to support city and infrastructure planning.
3. **Agriculture:** separating annual crops, permanent crops and pasture to support crop-area estimation and farm management.

4. **Environmental Monitoring:** detecting forests, rivers and lakes to help monitor deforestation, water resources and ecosystem change.
5. **Disaster Response and Risk Assessment:** quickly categorising areas to support flood, fire and damage assessment.
6. **Research and Education:** providing a clean reference implementation for studying Vision Transformers, transfer learning and model explainability.

By providing a fast, accurate and easily deployable solution, the project forms a foundation for geospatial analytics and general image-classification applications.

---

2. LITERATURE SURVEY

Image classification has evolved from hand-crafted features to deep learning. Early systems combined descriptors such as SIFT and HOG with classifiers such as Support Vector Machines, but they struggled with the variety of real-world images. Convolutional Neural Networks changed this: AlexNet (2012) won the ImageNet challenge, and deeper networks such as VGG and ResNet (He et al., 2016) used residual connections to train very deep models, making CNNs the standard for a decade. In parallel, the Transformer (Vaswani et al., 2017) replaced recurrence with self-attention in natural language processing. Dosovitskiy et al. (2021) introduced the Vision Transformer, which divides an image into 16 x 16 patches, embeds them linearly, adds positional information and feeds them to a standard Transformer encoder; pretrained on large datasets, it matched or exceeded CNNs while using less compute to train. Touvron et al. (2021) proposed DeiT, which made ViTs trainable on ImageNet-scale data using strong augmentation and distillation, and Steiner et al. (2022) showed that data augmentation and regularisation can substitute for additional data. Later designs such as the Swin Transformer (Liu et al., 2021) introduced hierarchical, shifted-window attention, and masked autoencoders (He et al., 2022) enabled self-supervised pretraining.

Several training techniques improved robustness: Mixup (Zhang et al., 2018) and CutMix (Yun et al., 2019) blend training images and labels, RandAugment (Cubuk et al., 2020) simplifies automatic augmentation, stochastic depth (Huang et al., 2016) randomly skips residual blocks, and AdamW (Loshchilov and Hutter, 2019) decouples weight decay from the gradient update. For remote sensing, Helber et al. (2019) released the EuroSAT benchmark for land-use and land-cover classification from Sentinel-2 imagery. For interpretability, Grad-CAM (Selvaraju et al., 2017) produces heat-maps showing which image regions influenced a prediction. Open-source libraries such as PyTorch, timm and ONNX Runtime have made these techniques easy to combine and deploy. Alongside

technical progress, concerns about dataset bias remain: a model trained on imagery from one region or sensor may perform worse elsewhere, which underlines the need for careful evaluation and transparency.

---

3. THEORETICAL ANALYSIS

3.1 BLOCK DIAGRAM

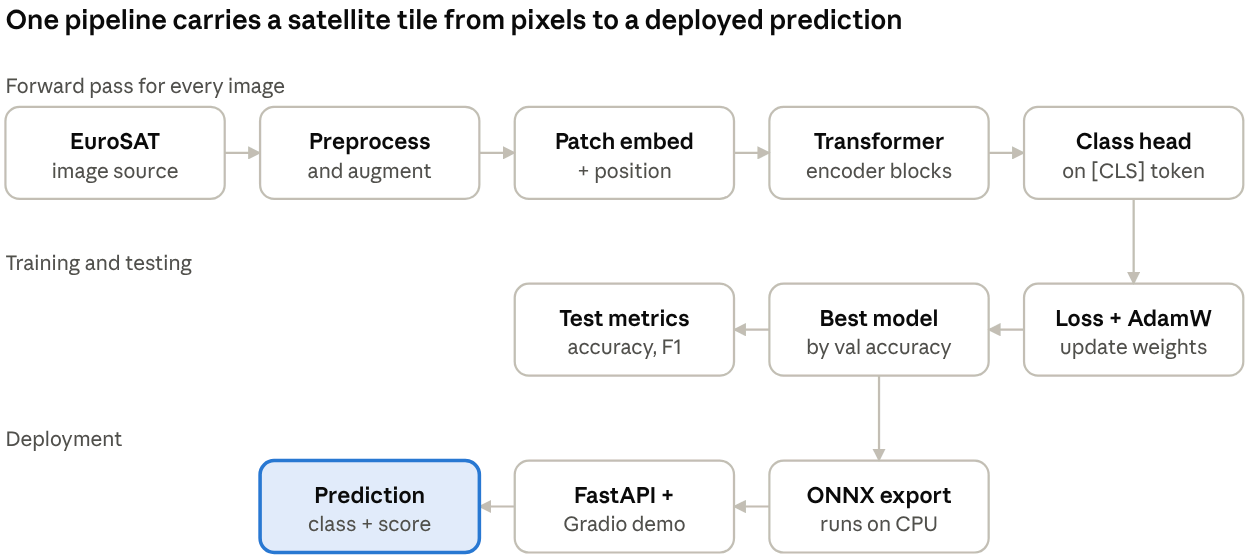


Figure 3.1 Block diagram

The system works in three connected stages. In the **training stage**, images from the training split are preprocessed and augmented, converted to patch embeddings, and passed through the Transformer encoder; the classification head produces class scores, the loss is computed and the optimizer updates the weights. The checkpoint with the best validation accuracy is saved. In the **testing stage**, the best checkpoint is loaded and evaluated on a held-out test split, producing accuracy, macro F1-score, a per-class report and a confusion matrix. In the **deployment stage**, the model is exported to ONNX and wrapped by a FastAPI service and a Gradio interface, which recognise the land-use class of an uploaded image and return the top predictions with confidence scores.

3.2 Hardware / Software Designing

HARDWARE REQUIREMENTS:

- **System**: Intel Core i5 / AMD Ryzen 5 or higher
- **GPU (for training)**: NVIDIA GPU with 8 GB VRAM or more, or the free Google Colab T4 GPU. Inference runs on a CPU.
- **Hard Disk**: 10 GB or more of free space
- **Monitor**: Full HD display or higher
- **Mouse**: Any standard mouse
- **RAM**: 8 GB (recommended: 16 GB or more)

SOFTWARE REQUIREMENTS:

- **Operating System**: Windows 10/11, Ubuntu 20.04 or above, or macOS
- **Backend Development Language**: Python 3.10 or above
- **Frameworks**: PyTorch for model development and training; FastAPI for the REST API
- **Libraries**:
  - torchvision: EuroSAT dataset and image transforms
  - timm: pretrained ViT / ResNet models, Mixup/CutMix and loss functions
  - scikit-learn: stratified splitting and evaluation metrics
  - NumPy and Matplotlib: numerical operations and plots
  - pytorch-grad-cam: Grad-CAM explainability
  - ONNX and ONNX Runtime: model export and fast CPU inference
  - Pillow: image handling; Gradio: front-end interface
  - TensorBoard: training curves; pytest: automated tests
- **IDE/Editor**: VS Code, Jupyter Notebook or Google Colab

WORKFLOW:

Step 1: Load the Dataset - EuroSAT is downloaded automatically and divided into stratified train, validation and test splits (70 / 15 / 15 percent).

Step 2: Preprocess and Augment Images - Images are resized, randomly cropped, flipped, transformed with RandAugment, normalised and randomly erased; Mixup/CutMix is applied to each training batch.

Step 3: Train the Models - The from-scratch ViT, the fine-tuned ViT-S/16 and the ResNet-50 baseline are trained from YAML configuration files; the best checkpoint is kept.

Step 4: Evaluate - Accuracy, macro F1-score, a per-class report and a confusion matrix are computed on the unseen test set.

Step 5: Explain Predictions - Grad-CAM generates a heat-map showing which regions of the image influenced the predicted class.

Step 6: Export and Deploy - The model is exported to ONNX, verified numerically against PyTorch, benchmarked, and served through FastAPI and Gradio.

---

4. EXPERIMENTAL INVESTIGATION

4.1 Background

1. PyTorch: PyTorch is an open-source deep-learning framework known for its dynamic computation graph, GPU acceleration and automatic differentiation. Its flexibility makes it the standard choice for research and increasingly for production. In this project PyTorch is used to implement the Vision Transformer from scratch (patch embedding, multi-head self-attention, MLP blocks and stochastic depth) and to run the training loop with mixed-precision support.

2. timm (PyTorch Image Models): timm is a library of pretrained computer-vision models and training utilities. It provides the pretrained ViT-S/16 weights (ImageNet-21k pretraining, ImageNet-1k fine-tuning) and the ResNet-50 baseline, along with Mixup/CutMix and loss functions. Using timm allows the project to compare a from-scratch model against transfer learning with minimal code changes.

3. EuroSAT Dataset: EuroSAT is a land-use and land-cover benchmark built from Sentinel-2 satellite images covering European cities. The RGB version used here contains 27,000 labelled 64 x 64 pixel tiles in ten classes with about 2,000 to 3,000 images per class. It is small enough to train on a free GPU yet realistic enough to be a meaningful geospatial task.

4. ONNX Runtime and FastAPI: ONNX (Open Neural Network Exchange) is a framework-independent model format, and ONNX Runtime executes it efficiently on CPUs. FastAPI is a modern Python web framework that builds REST APIs with automatic validation and interactive documentation. Together they allow the trained model to be served without installing PyTorch.

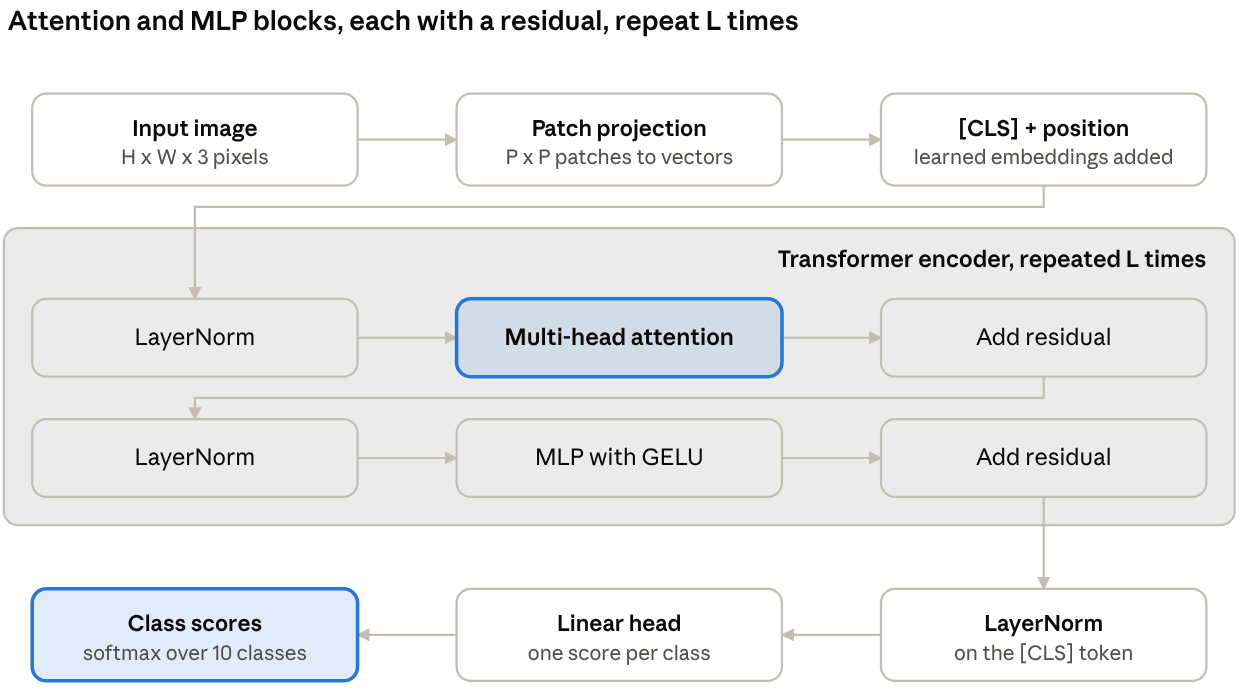
5. Gradio: Gradio is an open-source library for building web interfaces for machine-learning models. In this project it provides a simple page where a user uploads a satellite tile and immediately sees the predicted land-use class with its confidence.

The Vision Transformer is based on three critical tasks:

1. Patch Embedding: The input image is divided into non-overlapping square patches (for example 16 x 16 pixels). Each patch is flattened and linearly projected to an embedding vector; in the code this is a strided convolution. A learnable [CLS] token is prepended to the sequence, and learnable positional embeddings are added so that the model knows where each patch came from.

2. Transformer Encoding: The token sequence passes through a stack of encoder blocks. Each block applies Layer Normalisation, multi-head self-attention and a residual connection, followed by Layer Normalisation, a two-layer MLP with GELU activation and another residual connection. Self-attention lets every patch exchange information with every other patch, so global context is captured from the first layer.

3. Classification: After the final encoder block, the output of the [CLS] token is normalised and passed to a linear layer that produces one score per class. A softmax converts the scores to probabilities, and the class with the highest probability is the prediction.



ViT architecture · input to class scores

Figure 4.1.1 Vision Transformer architecture

4.2 Theoretical Analysis

1. Patch Sequence: For an image of height H, width W and C channels with patch size P, the number of patches is:

$N = (H \times W) / P^2$

For the from-scratch model (64 x 64 input, P = 8) N = 64 patches; for ViT-S/16 (224 x 224 input, P = 16) N = 196 patches. One extra [CLS] token is added in both cases.

2. Embedding and Encoder: The input sequence is:

$z_0 = [\, x_cls\, ;\, x_p^1 E\, ;\, x_p^2 E\, ;\, \ldots\, ;\, x_p^N E\,] + E_pos$

where E is the linear projection matrix and E_pos is the positional embedding. Each encoder block l = 1 … L computes:

$z'_l = MSA(\, LN(\, z_{l-1}\,)\,) + z_{l-1}$

$z_l = MLP(\, LN(\, z'_l\,)\,) + z'_l$

where MSA is multi-head self-attention and LN is Layer Normalisation. The attention operation is:

$Attention(Q, K, V) = softmax(\, Q\, K^T / \sqrt{d_k}\,)\, V$

The prediction is made from the [CLS] output: $\hat{y} = softmax(\, W \cdot LN(\, z_L^0\,)\,)$.

3. Training Objective and Regularisation: Mixup creates blended training samples:

$\tilde{x} = \lambda\, x_i + (1 - \lambda)\, x_j\,,\ \tilde{y} = \lambda\, y_i + (1 - \lambda)\, y_j\,,\ \lambda \sim Beta(\alpha, \alpha)$

CutMix pastes a rectangular region of one image into another and mixes the labels in proportion to the pasted area. The loss is the cross-entropy between the soft targets and the predicted probabilities:

$L = -\, \Sigma\, \tilde{y}_c\, \log p_c$

Stochastic depth randomly drops residual branches during training with a probability that increases with depth, and label smoothing (0.1) softens the targets.

4. Optimisation: The AdamW optimizer is used with weight decay 0.05, applied only to weight matrices (not biases, normalisation parameters, [CLS] token or positional embeddings). After a linear warm-up, the learning rate follows cosine decay:

$\eta(t) = \eta_min + \tfrac{1}{2}\, (\, \eta_max - \eta_min\,)\, (\, 1 + \cos(\, \pi\, t / T\,)\,)$

Gradients are clipped to a norm of 1.0, and mixed-precision (bfloat16 / float16) training is used on GPUs to reduce memory and time.

5. Evaluation Metrics:

$Accuracy = correct\ predictions\ /\ total\ predictions$

$Macro\ F1 = (1/K)\ \Sigma\, F1_k\,,\ where\ F1_k = 2\, P_k\, R_k\, /\, (\, P_k + R_k\,)$

with precision P_k and recall R_k for each of the K = 10 classes. A confusion matrix shows which classes are mistaken for each other.

6. Explainability (Grad-CAM): For a target class c, Grad-CAM weights each feature map A^k of the last encoder block by the average gradient of the class score:

$\alpha_k = (1/Z)\ \Sigma_i \Sigma_j\ \partial y^c / \partial A^k_{ij}$, $L_{Grad\text{-}CAM} = ReLU(\ \Sigma_k\ \alpha_k\ A^k\ )$

For a ViT, the patch tokens are reshaped back into a spatial grid so that the heat-map can be overlaid on the image.

7. Experimental Configuration:

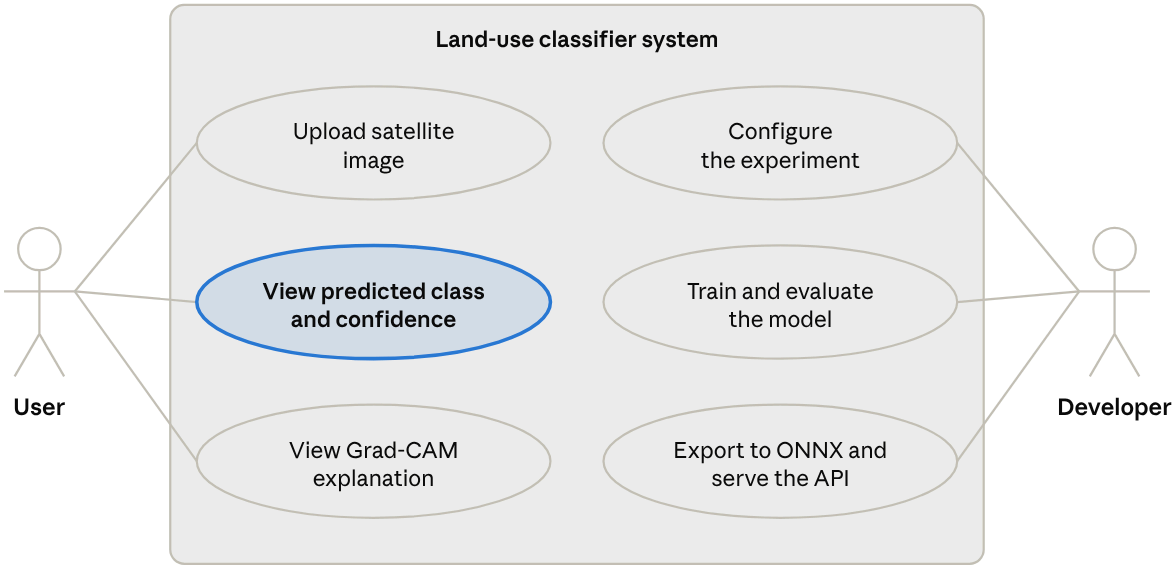
Setting	ViT (scratch)	ViT-S/16 (fine-tuned)	ResNet-50 (baseline)
Weights	Random initialisation	ImageNet-21k to 1k pretrained	ImageNet-1k pretrained
Input size	64 x 64	224 x 224	224 x 224
Patch size	8	16	not applicable
Embedding dim / depth / heads	192 / 6 / 3	384 / 12 / 6	not applicable
Epochs	80	10	10
Batch size	128	64	64
Learning rate	1e-3	5e-5	2e-4
Weight decay	0.05	0.05	0.05
Warm-up epochs	5	1	1
Mixup / CutMix alpha	0.2 / 1.0	0.2 / 1.0	0.2 / 1.0
Label smoothing	0.1	0.1	0.1
Stochastic depth	0.1	0.1	0.05

---

5. FLOWCHARTS

5.1 Use Case Diagram

Two actors reach the classifier through six use cases

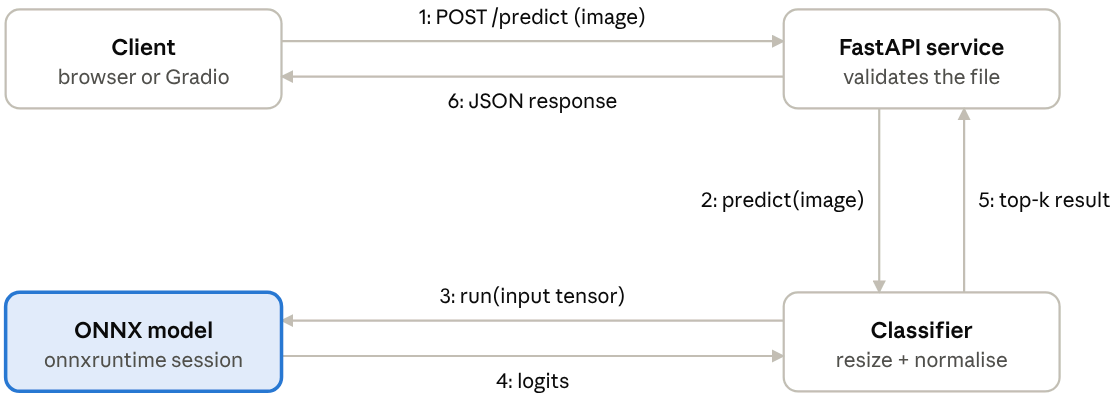


use case diagram · 2 actors, 6 use cases

Figure 5.1 Use case diagram

5.2 Collaboration Diagram

A prediction request takes six messages across four objects



collaboration diagram · 4 objects, 6 messages

Figure 5.2 Collaboration diagram

---

6. RESULT

The trained models are used through a web interface built with Gradio, and through the REST endpoint. The steps below describe how a prediction is made.

Step 1: Launch the interface. The page shows an upload area for a satellite tile and an empty output panel.

[Insert screenshot: Gradio interface before uploading an image]

Step 2: Upload an image, or drag and drop it into the input area.

[Insert screenshot: Gradio interface with an uploaded EuroSAT tile]

Step 3: The image is resized and normalised, and the ONNX model produces class probabilities.

Step 4: The interface shows the predicted land-use classes with their confidence scores.

[Insert screenshot: Output panel with the top-5 predicted classes and confidence]

Quantitative results (to be completed from the experiment output files runs/<experiment>/metrics.json):

Model	Parameters (M)	Test accuracy	Macro F1-score	CPU latency (ms, ONNX)
ViT (scratch)				
ViT-S/16 (fine-tuned)				
ResNet-50 (baseline)				

[Figure 6.1 Training and validation curves: insert the TensorBoard curves or plot history.json]

Figure 6.1 Training and validation curves

[Figure 6.2 Confusion matrix: insert runs/<experiment>/confusion_matrix.png]

Figure 6.2 Confusion matrix on the test set

[Figure 6.3 Grad-CAM: insert the overlay produced by scripts/predict.py with the --cam option]

Figure 6.3 Grad-CAM visualisation

---

7. ADVANTAGES & DISADVANTAGES

Advantages:-

1. **Global Context:** Self-attention lets every patch attend to every other patch, so the model captures long-range relationships that small convolution kernels see only in deeper layers.
2. **High Accuracy through Transfer Learning:** Fine-tuning a pretrained ViT reuses features learnt from millions of images, giving strong performance with short training.
3. **Explainable and Reproducible:** Grad-CAM shows why a prediction was made, and configuration files, fixed seeds and automated tests make experiments repeatable.
4. **Easy Deployment:** ONNX export lets the model run on a CPU without PyTorch, and FastAPI and Gradio give both an API and a user-friendly interface.

Disadvantages:-

1. **Data Hungry:** A ViT has less built-in image bias than a CNN, so a from-scratch model needs more data and strong augmentation to match a pretrained model.
2. **Computationally Expensive:** Self-attention cost grows with the square of the number of patches, so large images need a GPU and more memory.
3. **Dataset Bias:** EuroSAT covers European regions, RGB bands and low resolution, so accuracy may drop on other regions, sensors or seasons without fine-tuning.

---

8. APPLICATIONS

- ➢ **Land-use and Land-cover Mapping:** automatically categorising satellite tiles into residential, industrial, agricultural and natural areas.
- ➢ **Urban Planning:** monitoring city growth and infrastructure such as highways and industrial zones.
- ➢ **Agriculture:** estimating crop and pasture areas to support food-security planning.
- ➢ **Environmental Monitoring:** tracking forests, rivers and lakes to observe deforestation and water resources.

- ➢ **General Image Classification:** the same pipeline can be retrained for medical images, industrial inspection, wildlife monitoring or any other labelled image dataset.

---

9. CONCLUSION

The project demonstrates how the Vision Transformer, which treats an image as a sequence of patches, can be developed into a complete and professional machine-learning system. It combines a from-scratch implementation, which shows how every component works, with transfer learning from pretrained weights and a CNN baseline for comparison. Modern regularisation and optimisation techniques, systematic evaluation with accuracy, macro F1-score and confusion matrices, Grad-CAM explanations, ONNX export and a REST API with a Gradio interface turn a single notebook-style experiment into a deployable application. Future improvements could include newer architectures, multispectral input and optimisation for edge devices.

---

10. FUTURE SCOPE

Improved Models: Adopting hierarchical or data-efficient architectures such as the Swin Transformer or DeiT, and self-supervised pretraining such as masked autoencoders or DINOv2, could further improve accuracy with less labelled data.

Multispectral Input: Using all 13 Sentinel-2 spectral bands instead of RGB only would give the model more information about vegetation, water and built-up areas.

Real-Time and Edge Deployment: Quantisation (INT8), pruning and TensorRT or ONNX optimisation could allow fast inference on edge devices and drones.

Geographic Generalisation: Training and testing on geographically diverse datasets, and measuring fairness across regions, would make the system more reliable worldwide.

---

11. BIBLIOGRAPHY

1. Dosovitskiy, A., et al. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale." ICLR 2021.
2. Vaswani, A., et al. "Attention Is All You Need." NeurIPS 2017.
3. He, K., Zhang, X., Ren, S., & Sun, J. "Deep Residual Learning for Image Recognition." CVPR 2016.

4. Touvron, H., et al. "Training Data-Efficient Image Transformers & Distillation Through Attention." ICML 2021.
5. Steiner, A., et al. "How to Train Your ViT? Data, Augmentation, and Regularization in Vision Transformers." TMLR 2022.
6. Helber, P., Bischke, B., Dengel, A., & Borth, D. "EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification." IEEE JSTARS, 2019.
7. Zhang, H., Cisse, M., Dauphin, Y., & Lopez-Paz, D. "mixup: Beyond Empirical Risk Minimization." ICLR 2018.
8. Yun, S., et al. "CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features." ICCV 2019.
9. Cubuk, E. D., et al. "RandAugment: Practical Automated Data Augmentation with a Reduced Search Space." CVPR Workshops 2020.
10. Loshchilov, I., & Hutter, F. "Decoupled Weight Decay Regularization." ICLR 2019.
11. Huang, G., et al. "Deep Networks with Stochastic Depth." ECCV 2016.
12. Selvaraju, R. R., et al. "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization." ICCV 2017.
13. Liu, Z., et al. "Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows." ICCV 2021.
14. He, K., et al. "Masked Autoencoders Are Scalable Vision Learners." CVPR 2022.

APPENDIX

A. SOURCE CODE

The complete project is organised as follows: configs/ (YAML experiments), src/vitclf/ (data, models, engine, explain and export packages), scripts/ (train, evaluate, predict, export), app/ (ONNX inference, FastAPI, Gradio) and tests/. The main files are listed below.

A.1 configs/vit_finetune.yaml

```
# Pretrained ViT-S/16 (ImageNet-21k -> 1k, augreg weights) fine-tuned on
EuroSAT
experiment: vit_s16_finetune
seed: 42
data:
  root: data
```

```
  img_size: 224
  num_workers: 4
  val_size: 0.15
  test_size: 0.15
  mean: [0.5, 0.5, 0.5]   # augreg ViT weights were trained with mean=std=0.5
  std: [0.5, 0.5, 0.5]
  ra_num_ops: 2
  ra_magnitude: 9
  random_erasing: 0.25
model:
  source: timm
  name: vit_small_patch16_224.augreg_in21k_ft_in1k
  pretrained: true
  drop_path: 0.1
train:
  epochs: 10
  batch_size: 64
  lr: 5.0e-5
  weight_decay: 0.05
  warmup_epochs: 1
  label_smoothing: 0.1
  mixup_alpha: 0.2
  cutmix_alpha: 1.0
  mix_prob: 0.5
  grad_clip: 1.0
  amp: true
  out_dir: runs
```

A.2 src/vitclf/models/vit_scratch.py

```
"""Vision Transformer implemented from scratch in PyTorch (An Image is Worth
16x16 Words).

Attribute names (patch_embed, cls_token, pos_embed, blocks[i].norm1, norm,
head) deliberately
match timm's ViT so Grad-CAM, export and training code work for both models
unchanged.
"""
from __future__ import annotations

import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
class PatchEmbed(nn.Module):
    """Split image into non-overlapping patches and linearly project (a
strided conv)."""

    def __init__(self, img_size: int, patch_size: int, in_chans: int, dim:
int):
        super().__init__()
        assert img_size % patch_size == 0, "img_size must be divisible by
patch_size"
        self.grid_size = (img_size // patch_size, img_size // patch_size)
        self.num_patches = self.grid_size[0] * self.grid_size[1]
        self.proj = nn.Conv2d(in_chans, dim, kernel_size=patch_size,
stride=patch_size)

    def forward(self, x: torch.Tensor) -> torch.Tensor:        # (B,C,H,W) ->
(B,N,D)
        return self.proj(x).flatten(2).transpose(1, 2)


class Attention(nn.Module):
    def __init__(self, dim: int, num_heads: int, attn_drop: float = 0.0,
proj_drop: float = 0.0):
        super().__init__()
        assert dim % num_heads == 0
        self.num_heads = num_heads
        self.attn_drop = attn_drop
        self.qkv = nn.Linear(dim, dim * 3)
        self.proj = nn.Linear(dim, dim)
        self.proj_drop = nn.Dropout(proj_drop)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        B, N, C = x.shape
        qkv = self.qkv(x).reshape(B, N, 3, self.num_heads, C //
self.num_heads)
        q, k, v = qkv.permute(2, 0, 3, 1, 4).unbind(0)
        x = F.scaled_dot_product_attention(
            q, k, v, dropout_p=self.attn_drop if self.training else 0.0
        )
        x = x.transpose(1, 2).reshape(B, N, C)
        return self.proj_drop(self.proj(x))
```

```
class Mlp(nn.Module):
    def __init__(self, dim: int, hidden: int, drop: float = 0.0):
        super().__init__()
        self.fc1 = nn.Linear(dim, hidden)
        self.act = nn.GELU()
        self.fc2 = nn.Linear(hidden, dim)
        self.drop = nn.Dropout(drop)

    def forward(self, x):
        return self.drop(self.fc2(self.drop(self.act(self.fc1(x)))))


class DropPath(nn.Module):
    """Stochastic depth: randomly skip a residual branch per sample during
training."""

    def __init__(self, p: float = 0.0):
        super().__init__()
        self.p = p

    def forward(self, x):
        if self.p == 0.0 or not self.training:
            return x
        keep = 1.0 - self.p
        mask = x.new_empty(x.shape[0], *([1] * (x.ndim -
1))).bernoulli_(keep)
        return x * mask / keep


class Block(nn.Module):
    """Pre-LN Transformer encoder block: x + MHSA(LN(x)); x + MLP(LN(x))."""

    def __init__(self, dim, num_heads, mlp_ratio, drop, attn_drop,
drop_path):
        super().__init__()
        self.norm1 = nn.LayerNorm(dim, eps=1e-6)
        self.attn = Attention(dim, num_heads, attn_drop, drop)
        self.norm2 = nn.LayerNorm(dim, eps=1e-6)
        self.mlp = Mlp(dim, int(dim * mlp_ratio), drop)
        self.drop_path = DropPath(drop_path)

    def forward(self, x):
```

```
        x = x + self.drop_path(self.attn(self.norm1(x)))
        x = x + self.drop_path(self.mlp(self.norm2(x)))
        return x


class VisionTransformer(nn.Module):
    def __init__(self, img_size=64, patch_size=8, in_chans=3, num_classes=10,
embed_dim=192,
                 depth=6, num_heads=3, mlp_ratio=4.0, drop_rate=0.0,
attn_drop=0.0,
                 drop_path=0.1):
        super().__init__()
        self.patch_embed = PatchEmbed(img_size, patch_size, in_chans,
embed_dim)
        n = self.patch_embed.num_patches
        self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
        self.pos_embed = nn.Parameter(torch.zeros(1, n + 1, embed_dim))
        self.pos_drop = nn.Dropout(drop_rate)
        dpr = torch.linspace(0, drop_path, depth).tolist()      # linearly
increasing depth decay
        self.blocks = nn.Sequential(*[
            Block(embed_dim, num_heads, mlp_ratio, drop_rate, attn_drop,
dpr[i])
            for i in range(depth)
        ])
        self.norm = nn.LayerNorm(embed_dim, eps=1e-6)
        self.head = nn.Linear(embed_dim, num_classes)

        nn.init.trunc_normal_(self.pos_embed, std=0.02)
        nn.init.trunc_normal_(self.cls_token, std=0.02)
        self.apply(self._init_weights)

    @staticmethod
    def _init_weights(m):
        if isinstance(m, nn.Linear):
            nn.init.trunc_normal_(m.weight, std=0.02)
            if m.bias is not None:
                nn.init.zeros_(m.bias)
        elif isinstance(m, nn.LayerNorm):
            nn.init.ones_(m.weight)
            nn.init.zeros_(m.bias)

    def forward_features(self, x):
```

```
        x = self.patch_embed(x)
        cls = self.cls_token.expand(x.shape[0], -1, -1)
        x = torch.cat([cls, x], dim=1) + self.pos_embed
        x = self.blocks(self.pos_drop(x))
        return self.norm(x)

    def forward(self, x):
        return self.head(self.forward_features(x)[:, 0])        # classify
from [CLS] token
```

A.3 src/vitclf/models/factory.py

```
"""One entry point to build any model from a config: scratch ViT, pretrained
timm ViT, or CNN baseline."""
from __future__ import annotations

import timm
import torch.nn as nn

from .vit_scratch import VisionTransformer


def build_model(cfg, num_classes: int) -> nn.Module:
    m = cfg.model
    if m.source == "scratch":
        return VisionTransformer(
            img_size=cfg.data.img_size, patch_size=m.patch_size,
num_classes=num_classes,
            embed_dim=m.embed_dim, depth=m.depth, num_heads=m.num_heads,
            mlp_ratio=m.mlp_ratio, drop_path=m.drop_path,
        )
    if m.source == "timm":
        return timm.create_model(
            m.name, pretrained=m.pretrained, num_classes=num_classes,
drop_path_rate=m.drop_path
        )
    raise ValueError(f"unknown model.source: {m.source}")
```

A.4 src/vitclf/data/datamodule.py

```
"""EuroSAT (RGB, 27k Sentinel-2 patches, 10 land-use classes): download,
stratified split, loaders."""
from __future__ import annotations

import numpy as np
import torch
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms as T
from torchvision.datasets import EuroSAT


class TransformedSubset(Dataset):
    """Subset of a PIL-returning dataset with its own transform (so
train/val/test differ)."""

    def __init__(self, base: Dataset, indices: np.ndarray, transform):
        self.base, self.indices, self.transform = base, indices, transform

    def __len__(self) -> int:
        return len(self.indices)

    def __getitem__(self, i: int):
        img, y = self.base[int(self.indices[i])]
        return self.transform(img), y


def make_splits(targets, val_size: float, test_size: float, seed: int):
    """Stratified train/val/test split -> three disjoint index arrays."""
    targets = np.asarray(targets)
    idx = np.arange(len(targets))
    trainval, test = train_test_split(idx, test_size=test_size,
stratify=targets, random_state=seed)
    rel_val = val_size / (1.0 - test_size)
    train, val = train_test_split(
        trainval, test_size=rel_val, stratify=targets[trainval],
random_state=seed
    )
    return train, val, test


def build_transforms(d, train: bool):
    """`d` is cfg.data. Train: crop/flip/RandAugment/erasing. Eval: resize +
```

```
normalise only."""
    norm = T.Normalize(mean=list(d.mean), std=list(d.std))
    if train:
        return T.Compose([
            T.RandomResizedCrop(d.img_size, scale=(0.6, 1.0),
                                interpolation=T.InterpolationMode.BICUBIC),
            T.RandomHorizontalFlip(),
            T.RandomVerticalFlip(),          # valid for top-down satellite
imagery
            T.RandAugment(num_ops=d.ra_num_ops, magnitude=d.ra_magnitude),
            T.ToTensor(),
            norm,
            T.RandomErasing(p=d.random_erasing),
        ])
    return T.Compose([
        T.Resize((d.img_size, d.img_size),
interpolation=T.InterpolationMode.BICUBIC),
        T.ToTensor(),
        norm,
    ])


def build_dataloaders(cfg):
    d, t = cfg.data, cfg.train
    base = EuroSAT(root=d.root, download=True)          # transform=None ->
PIL images
    classes = list(base.classes)
    tr, va, te = make_splits(base.targets, d.val_size, d.test_size, cfg.seed)

    pin = torch.cuda.is_available()
    common = dict(num_workers=d.num_workers, pin_memory=pin,
                  persistent_workers=d.num_workers > 0)
    loaders = {
        # drop_last keeps batch size even, which timm's Mixup/CutMix requires
        "train": DataLoader(TransformedSubset(base, tr, build_transforms(d,
True)),
                            batch_size=t.batch_size, shuffle=True,
drop_last=True, **common),
        "val": DataLoader(TransformedSubset(base, va, build_transforms(d,
False)),
                          batch_size=t.batch_size * 2, shuffle=False,
**common),
        "test": DataLoader(TransformedSubset(base, te, build_transforms(d,
```

```
False)),
                            batch_size=t.batch_size * 2, shuffle=False,
**common),
    }
    return loaders, classes
```

A.5 src/vitclf/data/augment.py

```
"""Batch-level regularisation: Mixup / CutMix and the matching loss."""
from __future__ import annotations

import torch.nn as nn
from timm.data import Mixup
from timm.loss import LabelSmoothingCrossEntropy, SoftTargetCrossEntropy


def build_mixup(t, num_classes: int):
    """`t` is cfg.train. Returns a timm Mixup callable or None if
disabled."""
    if t.mixup_alpha <= 0 and t.cutmix_alpha <= 0:
        return None
    return Mixup(
        mixup_alpha=t.mixup_alpha,
        cutmix_alpha=t.cutmix_alpha,
        prob=t.mix_prob,
        switch_prob=0.5,
        mode="batch",
        label_smoothing=t.label_smoothing,
        num_classes=num_classes,
    )


def build_criterion(t, use_mixup: bool) -> nn.Module:
    if use_mixup:
        return SoftTargetCrossEntropy()
    return LabelSmoothingCrossEntropy(smoothing=t.label_smoothing)
```

A.6 src/vitclf/engine/trainer.py

```
"""Training loop: AdamW + warmup/cosine schedule + AMP + grad clipping +
Mixup/CutMix + best-checkpointing."""
from __future__ import annotations
```

```
import json
import math
import time
from pathlib import Path

import torch
from torch.optim.lr_scheduler import LambdaLR
from torch.utils.tensorboard import SummaryWriter
from tqdm import tqdm

from ..config import to_dict
from ..data.augment import build_criterion, build_mixup
from ..utils import count_params
from .evaluate import evaluate, full_report


def build_optimizer(model, lr: float, weight_decay: float):
    """AdamW; no weight decay on biases, norms, cls token and positional
embeddings."""
    decay, no_decay = [], []
    for name, p in model.named_parameters():
        if not p.requires_grad:
            continue
        skip = p.ndim <= 1 or name.endswith(".bias") or "pos_embed" in name
or "cls_token" in name
        (no_decay if skip else decay).append(p)
    groups = [{"params": decay, "weight_decay": weight_decay},
              {"params": no_decay, "weight_decay": 0.0}]
    return torch.optim.AdamW(groups, lr=lr, betas=(0.9, 0.999))


def build_scheduler(optimizer, total_steps: int, warmup_steps: int,
min_ratio: float = 0.01):
    def lr_lambda(step: int) -> float:
        if step < warmup_steps:
            return (step + 1) / max(1, warmup_steps)
        progress = (step - warmup_steps) / max(1, total_steps - warmup_steps)
        return min_ratio + (1 - min_ratio) * 0.5 * (1 + math.cos(math.pi *
progress))
    return LambdaLR(optimizer, lr_lambda)


class Trainer:
```

```
    def __init__(self, model, loaders, classes, cfg, device):
        self.cfg, self.device, self.classes = cfg, device, classes
        self.model = model.to(device)
        self.train_loader, self.val_loader, self.test_loader = (
            loaders["train"], loaders["val"], loaders["test"])
        t = cfg.train
        self.mixup = build_mixup(t, len(classes))
        self.criterion = build_criterion(t, self.mixup is not None)
        self.optimizer = build_optimizer(self.model, t.lr, t.weight_decay)
        spe = len(self.train_loader)
        self.scheduler = build_scheduler(self.optimizer, t.epochs * spe,
int(t.warmup_epochs * spe))

        self.use_amp = bool(t.amp) and device.type == "cuda"
        self.amp_dtype = (torch.bfloat16 if self.use_amp and
torch.cuda.is_bf16_supported()
                          else torch.float16)
        self.scaler = torch.amp.GradScaler(
            "cuda", enabled=self.use_amp and self.amp_dtype == torch.float16)

        self.out_dir = Path(t.out_dir) / cfg.experiment
        self.out_dir.mkdir(parents=True, exist_ok=True)
        self.writer = SummaryWriter(str(self.out_dir / "tb"))
        self.history: list[dict] = []

    def _train_epoch(self, epoch: int) -> float:
        self.model.train()
        total, n = 0.0, 0
        bar = tqdm(self.train_loader, desc=f"epoch {epoch}", leave=False)
        for x, y in bar:
            x, y = x.to(self.device, non_blocking=True), y.to(self.device,
non_blocking=True)
            if self.mixup is not None:
                x, y = self.mixup(x, y)                    # y becomes soft
targets
            with torch.autocast(self.device.type, dtype=self.amp_dtype,
enabled=self.use_amp):
                loss = self.criterion(self.model(x), y)
            self.optimizer.zero_grad(set_to_none=True)
            self.scaler.scale(loss).backward()
            if self.cfg.train.grad_clip:
                self.scaler.unscale_(self.optimizer)
                torch.nn.utils.clip_grad_norm_(self.model.parameters(),
```

```
self.cfg.train.grad_clip)
            self.scaler.step(self.optimizer)
            self.scaler.update()
            self.scheduler.step()
            total += loss.item() * x.size(0)
            n += x.size(0)
            bar.set_postfix(loss=f"{total / n:.4f}")
        return total / n

    def fit(self) -> None:
        print(f"model params: {count_params(self.model):.2f}M | device:
{self.device}")
        best = 0.0
        for epoch in range(1, self.cfg.train.epochs + 1):
            t0 = time.time()
            train_loss = self._train_epoch(epoch)
            val = evaluate(self.model, self.val_loader, self.device,
self.use_amp, self.amp_dtype)
            row = {"epoch": epoch, "train_loss": train_loss, "val_loss":
val["loss"],
                   "val_acc": val["acc"], "lr":
self.optimizer.param_groups[0]["lr"],
                   "sec": round(time.time() - t0, 1)}
            self.history.append(row)
            for k in ("train_loss", "val_loss", "val_acc", "lr"):
                self.writer.add_scalar(k, row[k], epoch)
            print(" | ".join(f"{k}={v:.4f}" if isinstance(v, float) else f"
{k}={v}"
                             for k, v in row.items()))
            if val["acc"] > best:
                best = val["acc"]
                torch.save({"model": self.model.state_dict(), "config":
to_dict(self.cfg),
                            "classes": self.classes, "epoch": epoch,
"val_acc": best},
                           self.out_dir / "best.pt")
        (self.out_dir / "history.json").write_text(json.dumps(self.history,
indent=2))
        print(f"best val acc: {best:.4f}")

    def test(self) -> dict:
        ckpt = torch.load(self.out_dir / "best.pt", map_location=self.device,
weights_only=True)
```

```
        self.model.load_state_dict(ckpt["model"])
        return full_report(self.model, self.test_loader, self.device,
self.classes, self.out_dir,
                           self.use_amp, self.amp_dtype)
```

A.7 scripts/train.py

```
"""Train + evaluate:  python scripts/train.py --config
configs/vit_finetune.yaml [--set train.epochs=5]"""
import argparse

from vitclf.config import load_config
from vitclf.data.datamodule import build_dataloaders
from vitclf.engine.trainer import Trainer
from vitclf.models.factory import build_model
from vitclf.utils import get_device, seed_everything


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--config", required=True)
    ap.add_argument("--set", nargs="*", default=[], help="overrides, e.g.
train.epochs=5")
    args = ap.parse_args()

    cfg = load_config(args.config, args.set)
    seed_everything(cfg.seed)
    device = get_device()

    loaders, classes = build_dataloaders(cfg)
    model = build_model(cfg, num_classes=len(classes))
    trainer = Trainer(model, loaders, classes, cfg, device)
    trainer.fit()
    trainer.test()


if __name__ == "__main__":
    main()
```

A.8 src/vitclf/explain/gradcam.py

```
"""Grad-CAM for ViTs: which image regions drove the prediction (works for
scratch and timm ViTs)."""
from __future__ import annotations

from functools import partial

import numpy as np
import torch
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image
from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget


def _reshape_transform(tensor: torch.Tensor, grid: int) -> torch.Tensor:
    """(B, 1+N, D) token sequence -> (B, D, grid, grid) feature map (drops
the [CLS] token)."""
    x = tensor[:, 1:, :].reshape(tensor.size(0), grid, grid, tensor.size(2))
    return x.permute(0, 3, 1, 2)


def grad_cam_overlay(model, x: torch.Tensor, mean, std, target_class: int |
None = None):
    """x: normalised (1,3,H,W) tensor. Returns (overlay_uint8_HWC,
heatmap_HW)."""
    model.eval()
    grid = model.patch_embed.grid_size[0]
    layers = [model.blocks[-1].norm1]
    targets = [ClassifierOutputTarget(target_class)] if target_class is not
None else None
    with GradCAM(model=model, target_layers=layers,
                 reshape_transform=partial(_reshape_transform, grid=grid)) as
cam:
        heat = cam(input_tensor=x, targets=targets)[0]
    m = torch.tensor(mean).view(3, 1, 1)
    s = torch.tensor(std).view(3, 1, 1)
    rgb = (x[0].detach().cpu() * s + m).clamp(0, 1).permute(1, 2,
0).numpy().astype(np.float32)
    return show_cam_on_image(rgb, heat, use_rgb=True), heat
```

A.9 src/vitclf/export/onnx_export.py

```
"""Export to ONNX, verify numerically against PyTorch, and benchmark CPU
latency."""
from __future__ import annotations

import time

import numpy as np
import onnxruntime as ort
import torch


def export_onnx(model, img_size: int, path: str, opset: int = 17) -> None:
    model.eval().cpu()
    dummy = torch.randn(1, 3, img_size, img_size)
    torch.onnx.export(
        model, dummy, path,
        input_names=["image"], output_names=["logits"],
        dynamic_axes={"image": {0: "batch"}, "logits": {0: "batch"}},
        opset_version=opset, do_constant_folding=True, dynamo=False,
    )


def verify_and_benchmark(model, path: str, img_size: int, runs: int = 30) ->
dict:
    model.eval().cpu()
    sess = ort.InferenceSession(path, providers=["CPUExecutionProvider"])
    x = torch.randn(4, img_size, img_size, 3).permute(0, 3, 1,
2).contiguous()
    with torch.no_grad():
        ref = model(x).numpy()
    out = sess.run(None, {"image": x.numpy()})[0]
    max_diff = float(np.abs(ref - out).max())

    def timeit(fn) -> float:
        fn()                                            # warm-up
        t0 = time.perf_counter()
        for _ in range(runs):
            fn()
        return (time.perf_counter() - t0) / runs * 1000

    one = x[:1]
    with torch.no_grad():
```

```
        torch_ms = timeit(lambda: model(one))
    ort_ms = timeit(lambda: sess.run(None, {"image": one.numpy()}))
    return {"max_abs_diff": max_diff, "torch_ms": round(torch_ms, 2),
"onnx_ms": round(ort_ms, 2)}
```

A.10 app/inference.py

```
"""Lightweight ONNX Runtime inference (no PyTorch needed at serving time)."""
from __future__ import annotations

import json

import numpy as np
import onnxruntime as ort
from PIL import Image


class Classifier:
    def __init__(self, model_path: str, meta_path: str):
        self.session = ort.InferenceSession(model_path, providers=
["CPUExecutionProvider"])
        meta = json.load(open(meta_path))
        self.classes = meta["classes"]
        self.size = meta["img_size"]
        self.mean = np.array(meta["mean"], dtype=np.float32).reshape(1, 1, 3)
        self.std = np.array(meta["std"], dtype=np.float32).reshape(1, 1, 3)

    def _preprocess(self, img: Image.Image) -> np.ndarray:
        img = img.convert("RGB").resize((self.size, self.size),
Image.BICUBIC)
        arr = (np.asarray(img, dtype=np.float32) / 255.0 - self.mean) /
self.std
        return arr.transpose(2, 0, 1)[None]                    # (1,3,H,W)

    def predict(self, img: Image.Image, top_k: int = 3) -> list[dict]:
        logits = self.session.run(None, {"image": self._preprocess(img)})[0]
[0]
        e = np.exp(logits - logits.max())
        probs = e / e.sum()
        idx = probs.argsort()[::-1][:top_k]
        return [{"label": self.classes[i], "probability":
round(float(probs[i]), 4)} for i in idx]
```

A.11 app/api.py

```
"""REST API:  uvicorn app.api:app --port 8000   ->   POST /predict (multipart
image)"""
import io
import os

from fastapi import FastAPI, File, HTTPException, UploadFile
from PIL import Image, UnidentifiedImageError

from app.inference import Classifier

app = FastAPI(title="ViT Land-Use Classifier", version="1.0.0")
clf = Classifier(os.getenv("MODEL_PATH", "models/vit.onnx"),
                 os.getenv("META_PATH", "models/meta.json"))


@app.get("/health")
def health():
    return {"status": "ok", "classes": clf.classes}


@app.post("/predict")
async def predict(file: UploadFile = File(...), top_k: int = 3):
    if file.content_type not in {"image/jpeg", "image/png"}:
        raise HTTPException(status_code=415, detail="Upload a JPEG or PNG
image.")
    try:
        img = Image.open(io.BytesIO(await file.read()))
    except UnidentifiedImageError:
        raise HTTPException(status_code=400, detail="Could not decode
image.")
    return {"predictions": clf.predict(img, top_k=max(1, min(top_k,
len(clf.classes))))}
```